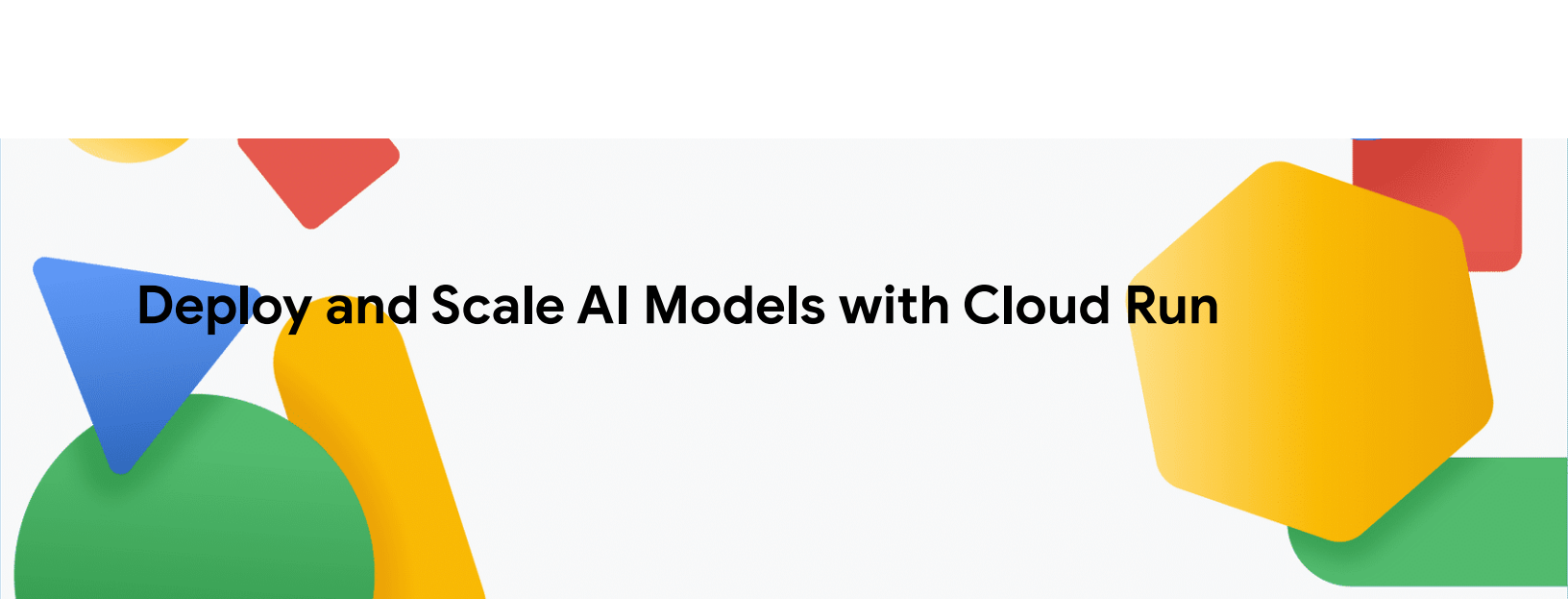




Deploy and Scale AI Models with Cloud Run



AI inference is the process of using a trained machine learning model to make predictions on new, unseen data by applying learned patterns. This course describes how you can use Cloud Run as a platform to run AI inference workloads by deploying and running large language models.

Click the **Start Course** button to begin.

A BRIEF INTRODUCTION TO CLOUD RUN

☰ What's in it for me?

☰ What is Cloud Run?

AI INFERENCE ON CLOUD RUN

☰ Use GPUs for AI Inference on Cloud Run

☰ Deploy lightweight language models on Cloud Run

☰ Optimize performance and cost-efficiency

☰ Integrate AI inference Services

☰



What did I walk away with?

What's in it for me?

AI inference is the process of using a trained machine learning model to make predictions on new, unseen data by applying learned patterns.

Welcome to this course on deploying and scaling AI models with Cloud Run. The course discusses how you can use Cloud Run to deploy services with GPUs that perform AI inference using large language models.

It is useful for those familiar with cloud-based serverless application deployment solutions, but who may not have experience with running AI inference using Google Cloud serverless products. The course includes demos that deploy a model for AI inference with GPUs and integrate gen AI apps with data storage services.

Here are the learning objectives of this course:



Course learning objectives

- ✓ Use Cloud Run GPUs for AI inference.
- ✓ Deploy lightweight language models on Cloud Run for AI inference.
- ✓ Optimize model deployments on Cloud Run for performance and cost efficiency.

- ✓ Integrate Cloud Run AI inference services with database services on Google Cloud.

What is Cloud Run?

Introduction to Cloud Run

In this module, we briefly introduce Cloud Run and its capabilities.

Cloud Run is a developer-focused, fully managed application platform to run code on Google's scalable infrastructure. Customers choose Cloud Run to increase innovation velocity and reduce their cost and risk.

Cloud Run is specifically designed to make developers productive and allow them to focus on solving business problems. As a serverless offering, Cloud Run abstracts away all infrastructure management, allowing developers to focus purely on writing code.

Cloud Run automatically scales container instances up or down based on traffic—even scaling to zero when idle—ensuring high availability and cost-efficiency by charging only for the compute resources consumed during request processing. This makes it an ideal solution for quickly and efficiently deploying web applications, APIs, and event-driven microservices.

Benefits of Cloud Run

Click each flashcard to learn more.

Increased productivity

Cloud Run allows developers to spend more time writing code and less time managing infrastructure.

Higher reliability

Cloud Run is redundant by default. Google is your Site Reliability Engineer.

Lower cost

Cloud Run autoscales to meet your needs and scales to zero. Pay only for what you use.

Ways to deploy

Cloud Run enables you to build and deploy containerized applications on Google Cloud Platform similar to how Google builds and deploys its own services. It leverages Cloud Build to build container images, and works well together with other services on Google Cloud, so you can build full-featured applications.

Here are ways to deploy your code to Cloud Run:

1

Container: Deploy a container image.

Supported repositories: Artifact Registry, Docker Hub

2

Repository: Continuously deploy code from a connected source Git repository.

Supported repository: GitHub

3

Source code: Deploy your code to Cloud Run with a single command.
Supported languages: Python, Node.js, Go, Java, Ruby, PHP, and .NET.

4

Function: Deploy single purpose functions.
Supported languages: Python, Node.js, Go, Java, Ruby, PHP, and .NET.

Use cases

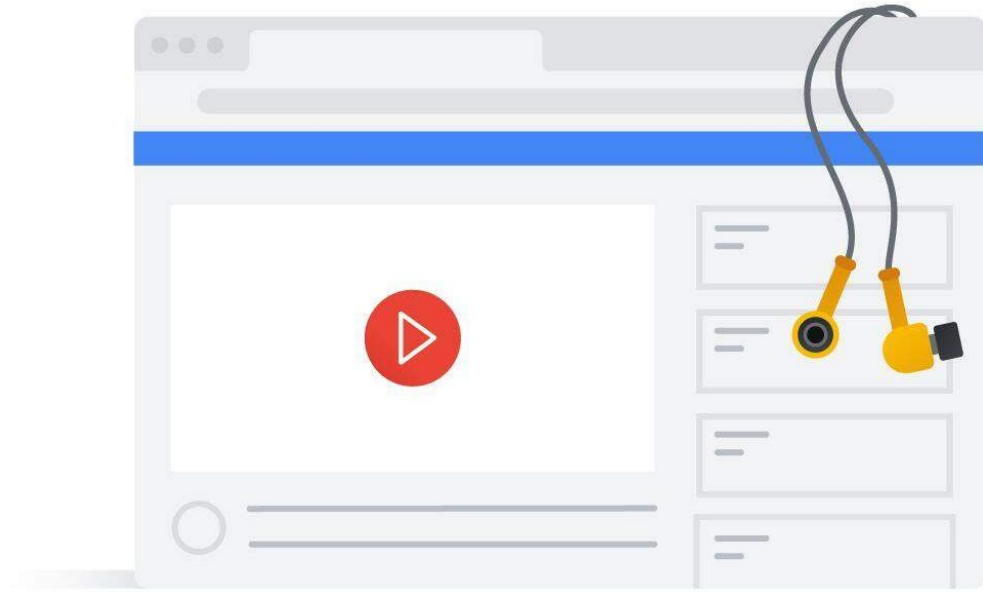
Here are some use cases for Cloud Run. Click each tab to learn more.

PUBLIC SERVICES

PRIVATE SERVICES

DATA PROCESSING

- Website.
- REST or GraphQL API.
- Streaming with WebSockets.
- AI agents.



PUBLIC SERVICES

PRIVATE SERVICES

DATA PROCESSING

- Internal website or API.
- Private HTTP or gRPC microservices.

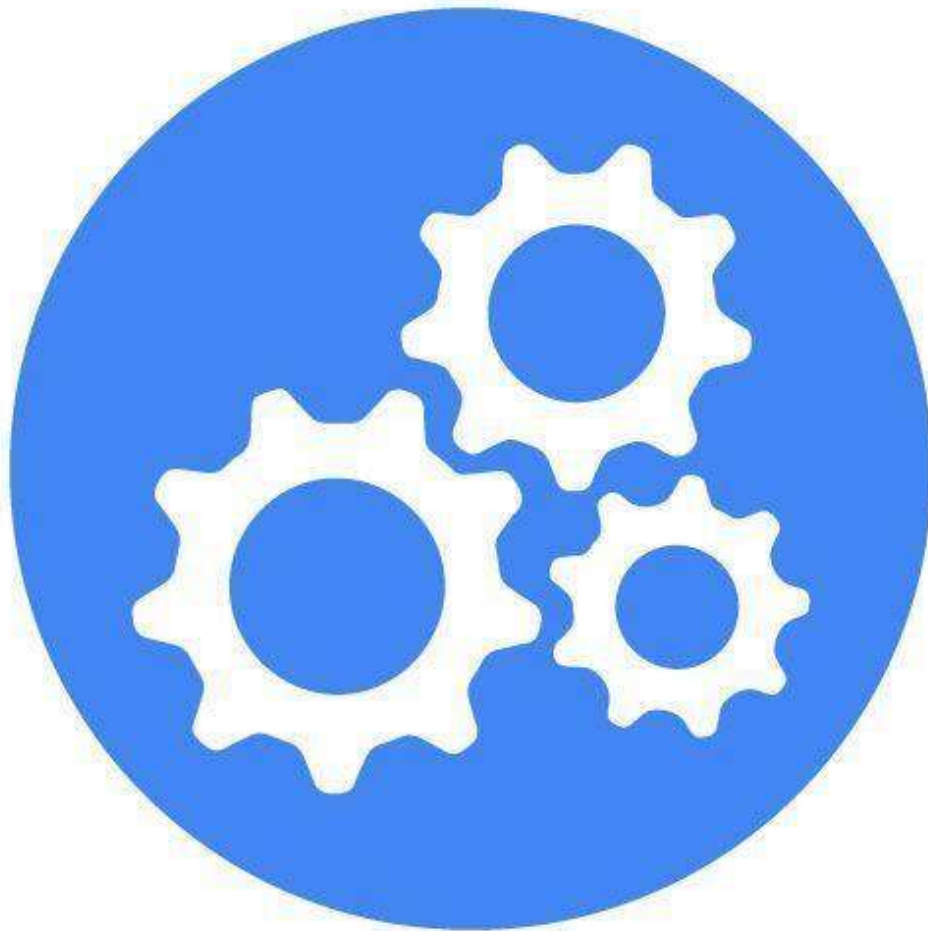


PUBLIC SERVICES

PRIVATE SERVICES

DATA PROCESSING

- Process queue messages.
- Event-driven architecture.
- Scheduled scripts.
- Background processing.
- Batch data processing.



Resources

There are two main resources for executing workloads in Cloud Run:

- A **Cloud Run service** is used to execute code that responds to web requests or events.
- A **Cloud Run job** is used to execute code that performs work (a job) and exits when finished.

Services

Serve HTTP traffic under a dedicated endpoint.

- Pay when code is processing requests.
- Use out-of-the-box URL with TLS.
- Leverage automatic scaling with minimum and maximum instances.
- Access a dedicated endpoint that is invoked by HTTPS requests, events, websockets, HTTP/2, gRPC.
- Employ built-in traffic splitting for gradual rollouts and rollbacks.
- Utilize revision history.

Jobs

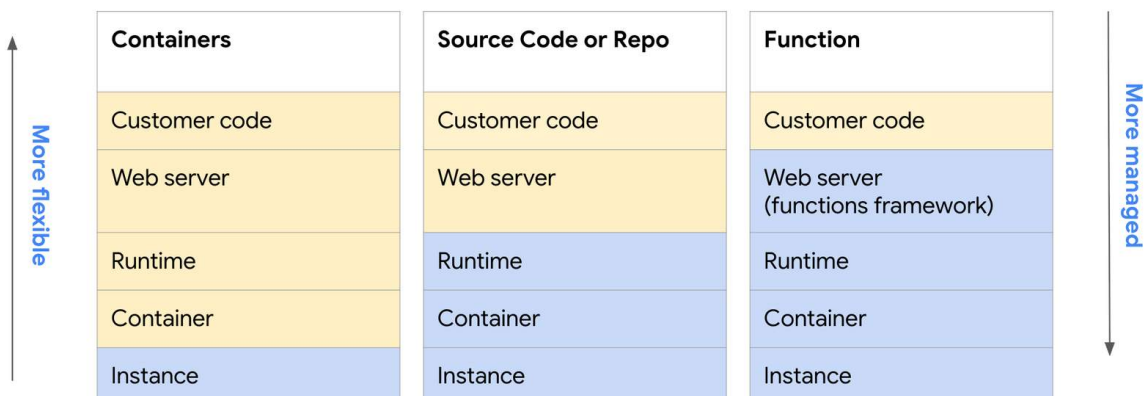
Execute tasks to completion.

- Are paid only while the job is executing.
- Do not have an HTTP endpoint.
- Have a maximum task timeout.
- Are executed manually or on a schedule.
- Run a specified number of tasks up to seven days.
- Have execution history.



Three ways to deploy and run a service

Managed by Google
Managed by you



- With containers, you have more flexibility but are responsible for creating the container image along with the runtime dependencies, and for responding to HTTP requests. Cloud Run manages creating and scaling the service instance.
 - When deploying from source code or a code repository, Cloud Run builds the container image for you and manages the service instances.
 - When using Cloud Run functions, you implement the function code that responds to an HTTP request or a Google Cloud event, while Cloud Run manages everything else.
-

Service trigger options

There are several ways we can trigger a Cloud Run service: HTTP (with an out-of-the-box TLS endpoint), events, on a schedule for Cloud Run jobs, or as part of a workflow.

TLS Endpoint	Events	On a schedule or part of a workflow
HTTP	Pub/Sub	Cloud Scheduler
Websockets	Google Cloud Storage	Workflows
HTTP/2	Many event sources with events from GCP services and third	
gRPC		

Knowledge check

Which of the following are primary reasons why a development team might choose Cloud Run for their application? (Select all that apply.)

- ☐ They want to deploy containerized applications without the burden of managing infrastructure or servers.
- ☐ Their application experiences variable traffic, and they want automatic scaling (including scaling down to zero) for cost-efficiency.
- ☐ They require direct SSH access and fine-grained control over the host operating system and kernel parameters.
- ☐ They need to run stateless web services, APIs, or event-driven functions packaged as containers.
- ☐ They are primarily looking to host a large, stateful database requiring persistent local disk attached directly to the running instance.

SUBMIT

Use GPUs for AI Inference on Cloud Run

Run AI inference workloads on Cloud Run with GPUs

To run real-time inference apps serving open generative AI models, you can use NVIDIA L4 GPUs on the Cloud Run platform.

Cloud Run features

Cloud Run is known for its simplicity, rapid autoscaling capabilities, and pay-per-use pricing. GPUs provide additional capabilities for your inference workloads and compute-intensive applications.

Click each card to learn more about these benefits.

Developer velocity

- Simple to use UI and CLI
- Fast deploy time
- No infrastructure management

Performance

- Low serving latency
- Rapid autoscaling
- Pre-installed GPU drivers

Cost efficiency

- Scale down to zero
- Pay-per-use pricing
- Fine-grained billing

Cloud Run GPU use cases

Here are a few use cases where you might consider using GPUs for your Cloud Run services.

Click each button to expand the items and learn more.

Real-time inference

- Perform real-time inference with lightweight open models (Gemma, Llama).
- Build custom chatbots, document summarization apps.
- Scale to handle spikes in user traffic.

Custom gen AI models

- Serve custom fine-tuned gen AI models, for example, to generate custom images.

- Scale down to optimize costs when the model is not in use.

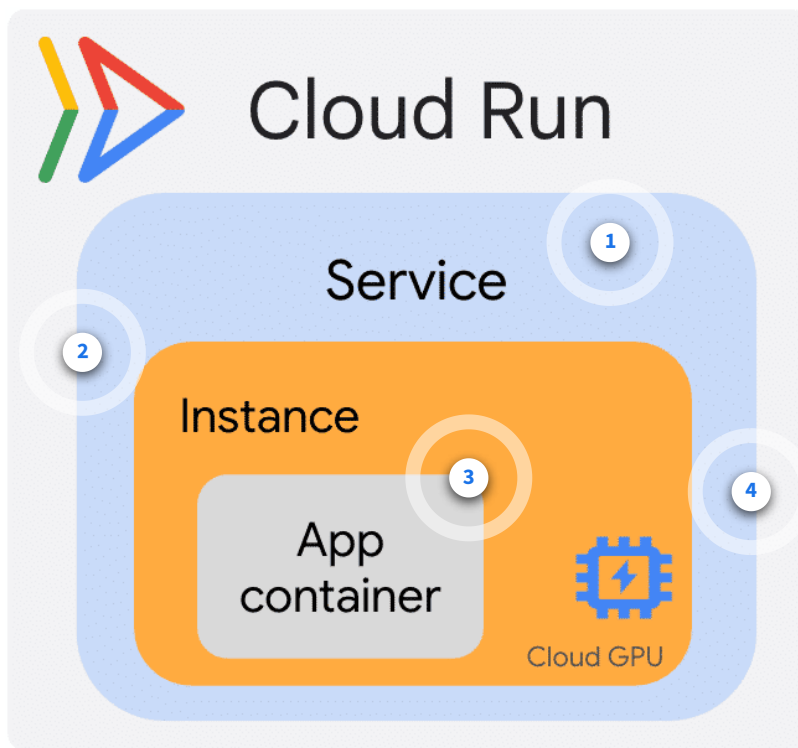
Compute intensive workloads

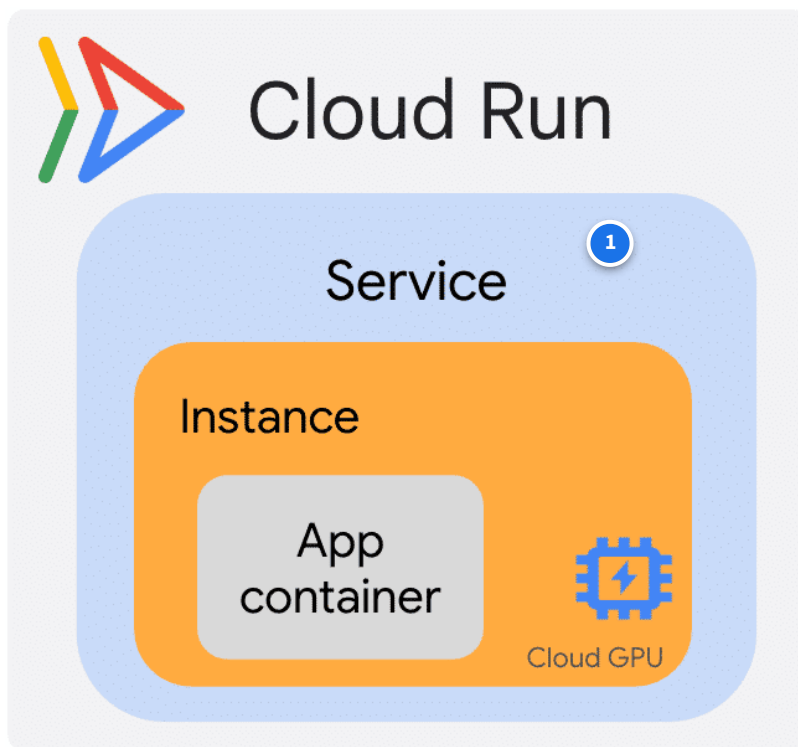
- Speed up compute-intensive services, such as image recognition.
- Perform video transcoding, streaming, and 3D rendering.

GPU support for Cloud Run

Here are some of the key features of Cloud Run GPUs as they apply to a service, instance, and app container in Cloud Run.

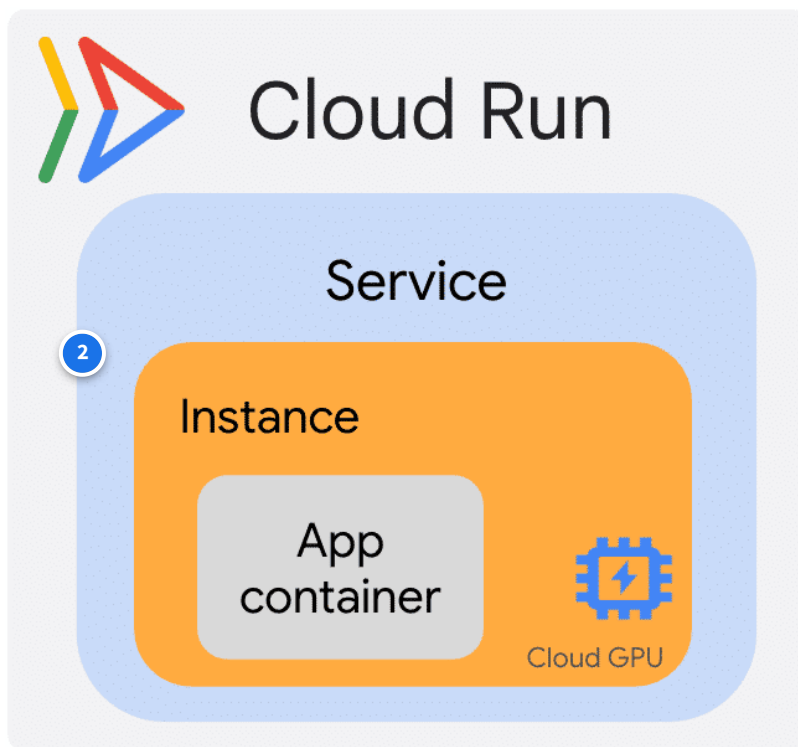
Click each marker to learn more.





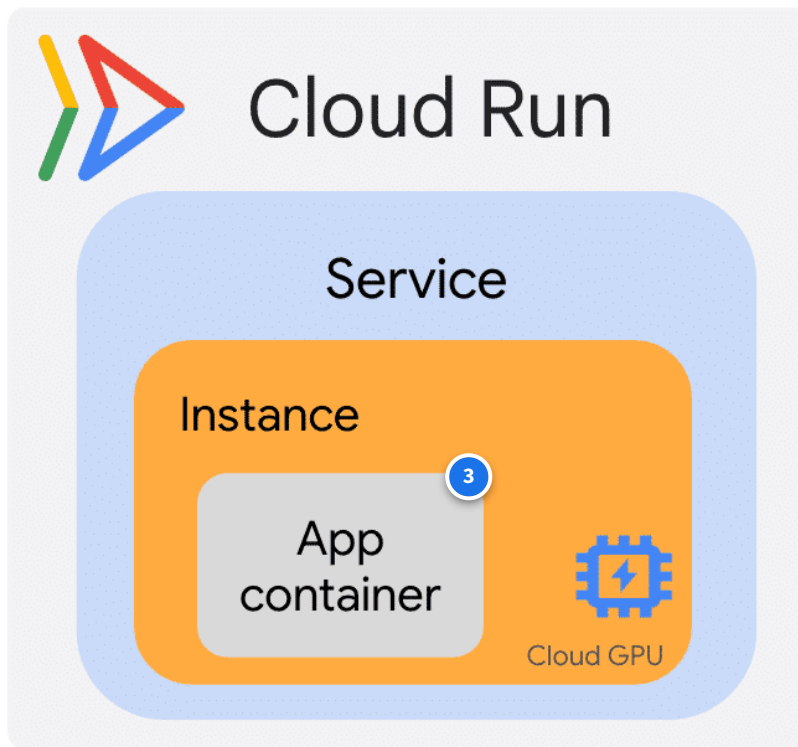
Cloud Run service

- A service provides an ingress endpoint to your gen AI app.
- To deploy a Cloud Run service with GPUs, use the `--gpu` flag in the CLI deploy command.



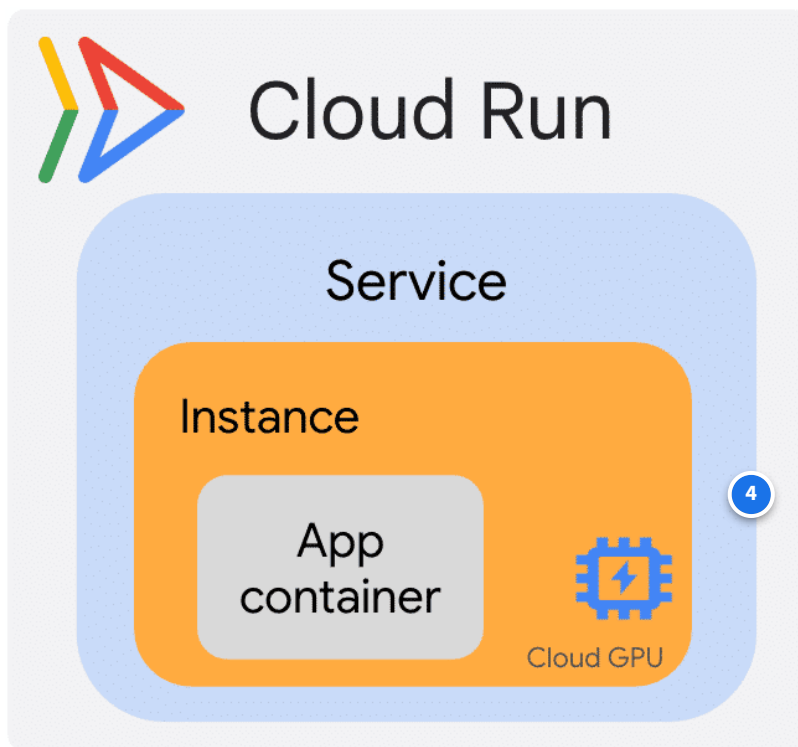
Instance

- Instances scale with traffic.
- You can scale to zero when the app is not in use.
- The minimum instance size is 4 vCPU and 16GB memory.
- You can have a maximum of 1 GPU per instance.
- The maximum number of GPUs is managed by quota per project and per region.



App container

- Serve LLMs with low latency and fast deployment.
- A GPU can only be attached to one container in the instance.



Cloud GPU

- Drivers and libraries are pre-installed.
- It uses L4 GPU with NVIDIA driver pre-installed.
- GPU on Cloud Run is fully managed and available on-demand.
- It is currently available in some regions and expanding.

Deploy a Cloud Run GPU

Let's review the process to deploy a Cloud Run service with GPU.

Prerequisites

Here are the prerequisites required to use GPUs in Cloud Run:

1. Select or create a Google Cloud project.
2. Enable billing for your Google Cloud project.
3. Enable the Cloud Run API.
4. Request a quota change (*Total Nvidia L4 GPU allocation*) for your Google Cloud project and region.

Click the Start or Next button (>) to continue.

Step 2

Set required roles

To configure and deploy a Cloud Run service, make sure you have the following roles:

1. *Cloud Run Developer* on the Cloud Run service.
2. *Service Account User* on the service identity.

Step 3

Create or edit a Cloud Run service

» Cloud Run

« Create service

Container image URL

us-docker.pkg.dev/cloudrun/container/hello

SELECT

TEST WITH A SAMPLE CONTAINER

Should listen for HTTP requests on \$PORT (default: 8080) and not rely on local state. [How to build a container?](#)

Configure

Service name *

hello

Some locations have been restricted due to a policy set by your organization. [Learn more about restricting locations.](#)

Region *

us-central1 (Iowa)

▼

How to pick a region?

Endpoint URL

https://hello-652546730991.us-central1.run.app

Authentication *

☐ Allow unauthenticated invocations

Check this if you are creating a public API or website.

☐ Require authentication

Manage authorized users with Cloud IAM.

Billing

☐ Request-based

Charged only when processing requests. CPU is limited outside of requests.

☒ Instance-based

Charged for the entire lifecycle of instances. Full CPU for the entire lifetime of each instance.

Service scaling

You can use the Google Cloud console, Google Cloud CLI, or YAML to configure GPU for your service.

1. Navigate to the Cloud Run page in the Cloud console.
2. To create a new service, click **Deploy Container**, and select **Service**. (To configure an existing service, select the service, and click **Edit and Deploy New Revision**).
3. For a new service provide the initial service configuration.
4. Set **Instance-based** billing for your service.

Step 4

Configure your Cloud Run service

Cloud Run < Create service

CONTAINER(S) VOLUMES NETWORKING SECURITY

^ Edit Container

Container image URL
us-docker.pkg.dev/cloudrun/container/hello

Container port
8080
Requests will be sent to the container on this port. We recommend listening on \$PORT instead of this specific number.

SETTINGS VARIABLES & SECRETS VOLUME MOUNTS

Container name: hello-1 EDIT

Container command
Leave blank to use the entry point command defined in the container image.

Container arguments
Arguments passed to the entry point command.

Resources

Memory
16 GiB
Memory to allocate to each instance of this container.

CPU
4
Number of vCPUs allocated to each instance of this container.

☒ GPU PREVIEW
Attach GPUs to this container

GPU type
NVIDIA L4

Number of GPUs
1

In the service configuration, expand **Container(s)**, **Volumes**, **Networking**, **Security**.

To use GPUs, configure your Cloud Run service with:

1. GPU type (for example NVIDIA L4) and number of GPUs.
2. A minimum of 4 CPUs. (8 CPUs recommended).
3. A minimum of 16 GiB memory (32 GiB recommended).
4. Optimal maximum concurrency based on your usage. (We review this topic later in the course.)
5. Maximum instances set to below the allowed GPU quota per project per region.
6. GPU redundancy set to Zonal Redundancy (if you have reserved GPU capacity across multiple zones), or to No Zonal Redundancy.

Configure additional settings for your service such as health check, execution environment, authentication, etc.

You can then click **Create** or **Deploy** to create or deploy the service.

View GPU settings

Cloud Run

Service details

EDIT & DEPLOY NEW REVISION

SET UP CONTINUOUS DEPLOYMENT

TEST

LEARN

hello

Region: us-central1

URL: https://hello-652546730991.us-central1.run.app

Scaling: Auto (Min: 0)

METRICS

SLOS

LOGS

REVISIONS

TRIGGERS

NETWORKING

SECURITY

YAML

Revisions

MANAGE TRAFFIC

Filter

Filter revisions

Name	Traffic	Deployed	Revision tags	Actions
hello-00001-n7q	100% (to latest)	25 minutes ago	+	

General

Billing

Instance-based

Startup CPU boost

Enabled

Concurrency

80

Request timeout

300 seconds

Execution environment

Second generation

Autoscaling

Revision max instances

3

Image

us-docker.pkg.dev/cloudrun/container/hello@sha256:...

Port

8080

Build

(no build information available)

Source

(no source information available)

Command and args

(container entrypoint)

CPU limit

4

Memory limit

16GiB

GPU

1 NVIDIA L4

PREVIEW

After the service is created and deployed, you can view the GPU settings in the console.

1. In Cloud Run, navigate to the **Service details** page of your service, and click the **Revisions** tab.
2. In the service details panel, click the **Containers** tab.

The service configuration along with GPU is displayed.

Knowledge check

What type of billing is required when configuring a Cloud Run service with GPU?

- ☐ Request-based billing.
- ☐ Instance-based billing.

SUBMIT

Deploy lightweight language models on Cloud Run

Let's review the process to deploy and run lightweight models on a GPU-enabled Cloud Run service. In this lesson, we'll discuss how to deploy [Gemma 3](#), an open language model, to Cloud Run. We'll also use [Ollama](#), a model inference server and tool to manage open models locally and handle prompts.

Google Cloud services

The process to deploy AI models on Cloud Run involves the use of these Google Cloud services with the required roles.

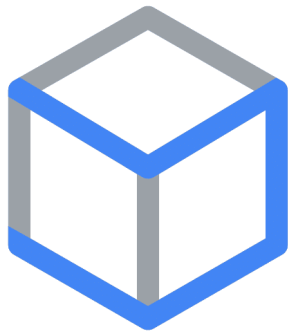
Click each card to learn more.



Cloud Build

Builds a container image with Ollama and the model files.

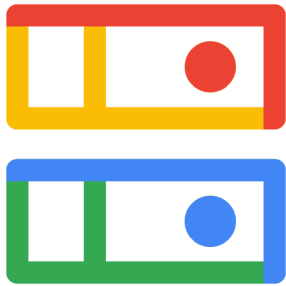
Required role: Cloud Build Editor



Artifact Registry

Hosts a Docker repository that stores a container image with Ollama and the model files.

Required role: Artifact Registry Admin



Cloud Storage

To build the Cloud Run service.

Required role: Cloud Storage Admin



Cloud Run

To deploy Ollama and the model as a Cloud Run service.

Required role: Cloud Run Admin

Deploy and run model inference

Introduction

Let's review the details on the steps to deploy and run a Cloud Run service for AI inferencing.

Note: The steps in this process use the Google Cloud CLI (gcloud) commands.

Step 2

Set your Google Cloud project and region

```
gcloud config set project PROJECT_ID  
gcloud config set run/region REGION
```

Set your Google Cloud project and region for your Cloud Run service.

Step 3

Enable required APIs

```
gcloud services enable \  
artifactregistry.googleapis.com \  
cloudbuild.googleapis.com \  
run.googleapis.com \  
storage.googleapis.com
```

Enable the Artifact Registry, Cloud Build, Cloud Run, and Cloud Storage APIs to store, build, and deploy your service to Cloud Run.

Step 4

Create a Docker repository in Artifact Registry

```
gcloud artifacts repositories create REPOSITORY \  
--repository-format=docker --location=REGION
```

To store the container image that has Ollama and the Gemma 3 model files, create a Docker repository in Artifact Registry.

Step 5

Write the Dockerfile to create a container image

```
FROM ollama/ollama:0.6.0

# Listen on all interfaces, port 8080
ENV OLLAMA_HOST 0.0.0.0:8080

# Store model weight files in /models
ENV OLLAMA_MODELS /models

# Reduce logging verbosity
ENV OLLAMA_DEBUG false

# Never unload model weights from the GPU
ENV OLLAMA_KEEP_ALIVE -1

# Store the model weights in the container image
ENV MODEL gemma3:4b
RUN ollama serve & sleep 5 && ollama pull $MODEL

# Start Ollama
ENTRYPOINT ["ollama", "serve"]
```

The [Dockerfile](#) is used to create a Docker container image with Ollama and Gemma 3.

- Before Ollama can start serving inference requests, it must load the model weights into GPU memory during container instance startup.
- To ensure a fast instance startup, it's recommended to store the model weight files directly in the container image.

Note: The Gemma 3 model weight files take up 8GB of storage. It might be impractical to store larger model weight files of other larger models in the container image.

Step 6

Build the container image with Cloud Build

```
gcloud builds submit --tag \
us-central1-docker.pkg.dev/PROJECT_ID/REPOSITORY/ollama-gemma \
--machine-type e2-highcpu-32
```

Build the container image with Cloud Build and push the image to the repository that you created earlier in Artifact Registry.

- You can also build the image locally with Docker and push it to Artifact Registry.
- To reduce build time, the *gcloud build* command uses a machine type with more CPU and network bandwidth.

Step 7

Deploy the container image to Cloud Run

```
gcloud run deploy ollama-gemma --image \
us-central1-docker.pkg.dev/PROJECT_ID/REPOSITORY/ollama-gemma \
--concurrency 4 \
--gpu 1 --gpu-type nvidia-l4 \
--max-instances 7 \
--cpu 8 --memory 32Gi \
--no-allow-unauthenticated \
--no-cpu-throttling \
--timeout=600 \
--set-env-vars OLLAMA_NUM_PARALLEL=4
```

You can now deploy the container image and make Ollama with Gemma 3 available as a Cloud Run service.

The Cloud Run service is configured with:

- One Nvidia L4 GPU per instance.
- A maximum of four concurrent requests to the instance that matches the number of request slots (OLLAMA_NUM_PARALLEL) available per model to serve concurrent inference requests.
- A maximum of seven Cloud Run service instances which should match your GPU quota per project and per region.
- The no-cpu-throttling setting required to use GPU.

Test the service

```
gcloud run services proxy ollama-gemma --port=9090
```

```
curl http://localhost:9090/api/generate -d \
'{"model": "gemma3:4b", "prompt": "Why is the sky blue?"}'
```

With the service deployed, you can test it by sending requests to the Cloud Run service endpoint.

Using the Cloud Run developer proxy, you can send a request with a valid authorization token to the service.

The video demonstrates the process outlined above to deploy an app to Cloud Run using a GPU for AI inference.

Click the play button to watch the video.

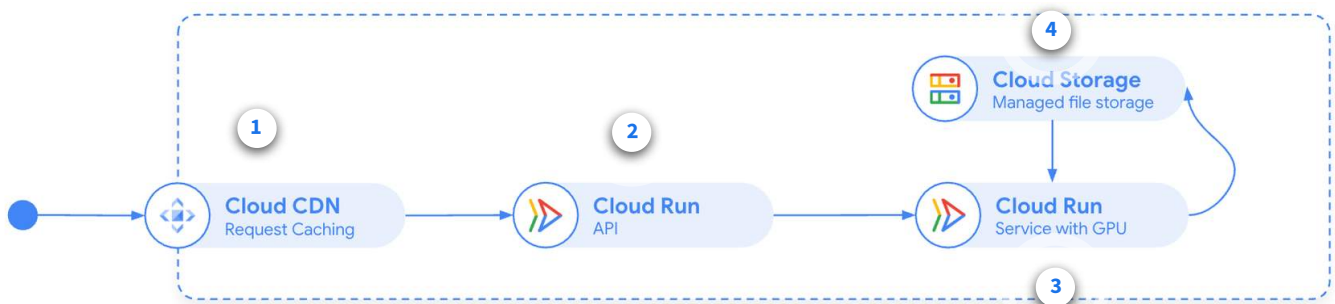
Use Cloud Run for AI Inference

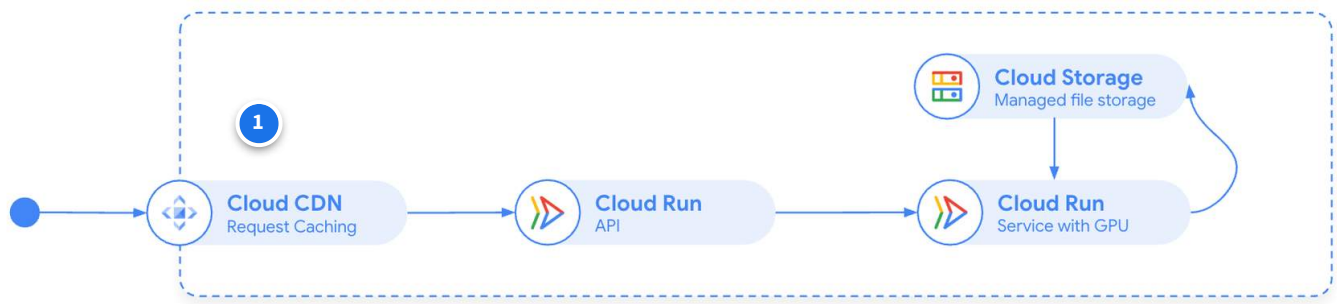


Design pattern with Cloud Storage

The previous process stored a lightweight model in the container image to reduce the container instance startup time. For larger models, a pattern for running AI inference on Cloud Run uses Cloud Storage as a data store for the model files.

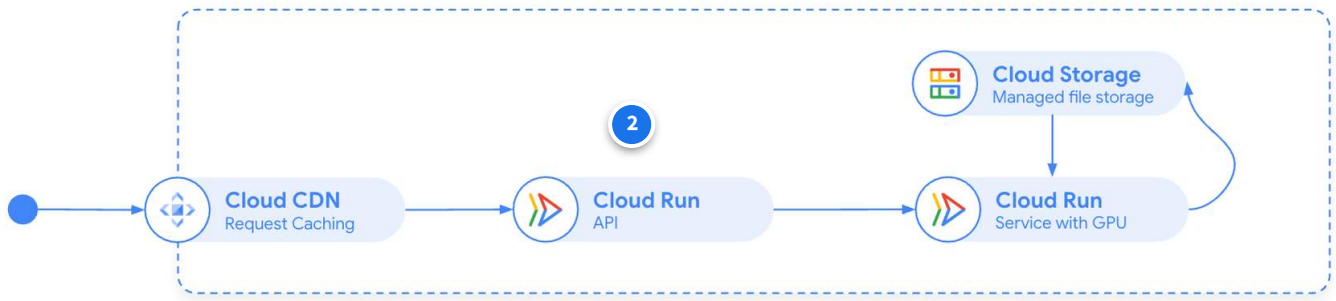
Click each marker to learn more.





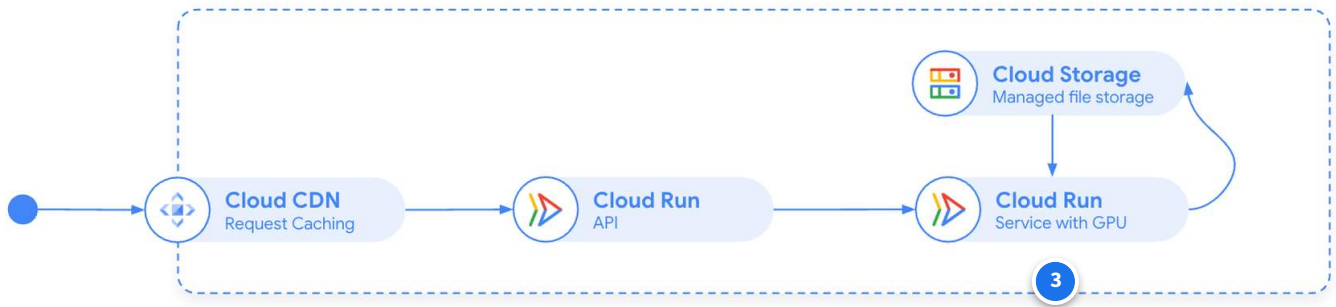
Cloud CDN (content delivery network)

- Use Google's global edge network to cache and serve content closer to users.



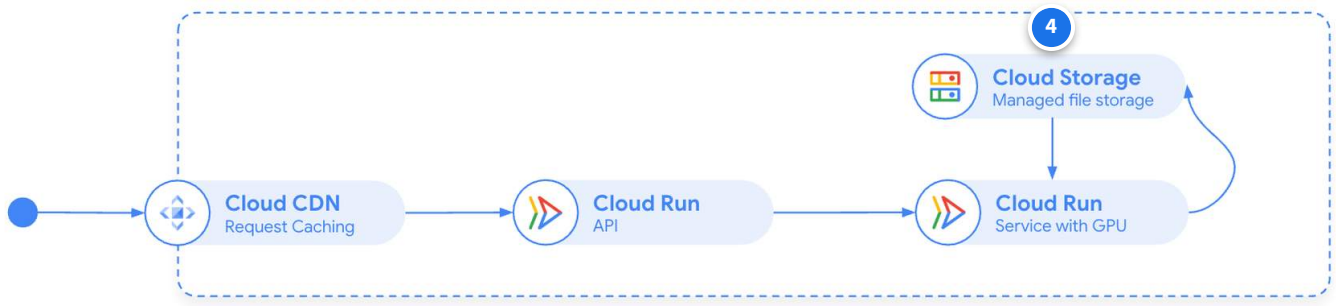
Cloud Run service API

- Use a Cloud Run service to provide a scalable API endpoint to serve requests.
- Run LangChain or custom code to validate requests, and orchestrate calls to LLMs.
- LangChain is a framework to build applications that integrate with LLMs and external data sources.



Cloud Run service with GPU

- Use GPUs with a Cloud Run service to perform AI inference such as text-to-text and image generation from text.
- The frontend Cloud Run service is used to send back the model response.



Cloud Storage

- Use Cloud Storage to store the model files for LLMs.
- The model is downloaded during container startup allowing for a faster image deployment to Cloud Run.

Optimize performance and cost-efficiency

Let's discuss some best practices to optimize performance and cost-efficiency when using a Cloud Run service for AI inference.

Storing and loading ML models

Use models that require minimal transformation into GPU-ready structures, and optimize how models are stored and loaded to minimize the time to load.

Here are some options to store the model and factors to consider. Click each tab to learn more.

CONTAINER IMAGE

CLOUD STORAGE

INTERNET

Option 1: Store the model in the container image

To reduce the time for container instance startup, you can store the model in the container image based on the amount of storage used by the model files. Consider these other factors:

- Deployment to Cloud Run might be slow depending on the model size.
- Changes to the container image will require redeployment.
- Fast container startup time improving service scale up.
- Cost of storage could be high with potential multiple copies in Artifact Registry.
- Benefits from Cloud Run's container image streaming to optimize file loading.



Container image

CONTAINER IMAGE

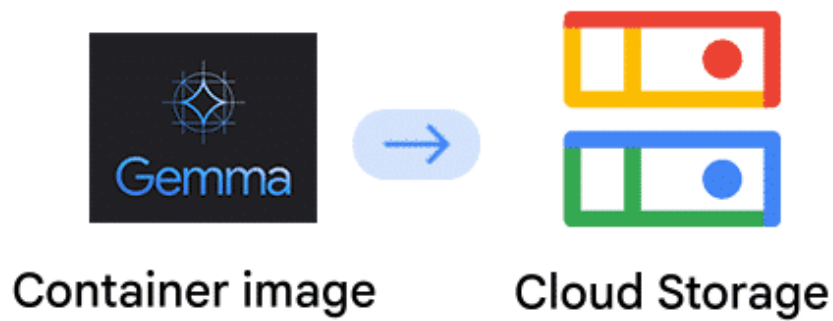
CLOUD STORAGE

INTERNET

Option 2: Store the model in Cloud Storage

For larger models, store the model files in Cloud Storage, while considering these factors:

- Deployment to Cloud Run is fast.
- The model is downloaded during container startup, and with network optimizations the startup time can be reduced.
- Does not require changes to the container image.
- Easy to set up when used with a Cloud Storage FUSE volume mount.
- When using the gcloud CLI, the model files are downloaded in parallel making it faster than the FUSE volume mount.
- To optimize loading the model from Cloud Storage, use Direct VPC with egress set to all-traffic, and set up Private Google Access.
- One copy of the model is stored in Cloud Storage reducing cost.



CONTAINER IMAGE

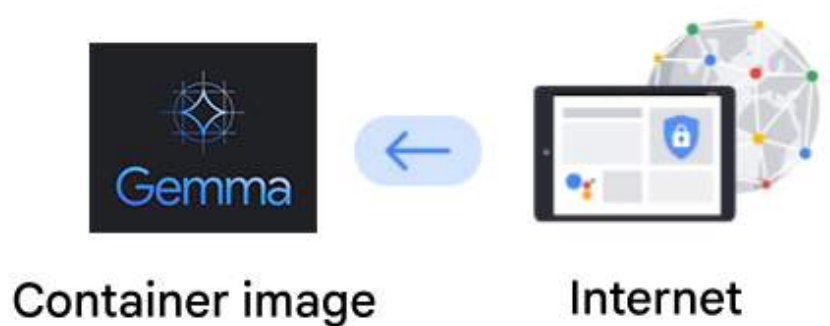
CLOUD STORAGE

INTERNET

Option 3: Download a model from the internet.

You can also use a model that is downloaded from the internet, with these considerations:

- Container image deployment to Cloud Run is fast.
- The model is downloaded during container startup which makes the startup time unpredictable. This is due to the possibility of the model host service being down or inaccessible, and model transformations applied by the framework during initialization.
- Cost will depend on the model hosting service or provider.
- To optimize loading the model from the internet, route all traffic through the VPC network, and set up Cloud NAT with high bandwidth.



Building the container image

There are also some best practices to consider when building your container image.

Click each button to expand the item and learn more.

Choose a good base image for your container.

Start with an image from [Deep Learning Containers](#) or the [NVIDIA container registry](#) for the machine learning (ML) framework you are using. These images come with the latest performance-related packages installed.

To maximize model concurrency, choose 4-bit quantized models.

Quantization produces models that are smaller and faster, reducing the amount of GPU memory required to serve the model, and increases parallelism at run time.

Pick a model format with fast load times.

To minimize container startup time and request latency, use a model format that accurately reflects the target quantization type, and require less transformations when loaded onto the GPU.

Create and warm model caches —

When building the container image, start the model on the build machine to enable prompt caching with common or example prompts for real-world use.

Deploying the container image

Set your Cloud Run service concurrency accurately

To control the maximum number of concurrent requests to your instance, use the **maximum concurrent requests per instance** setting for your Cloud Run service.

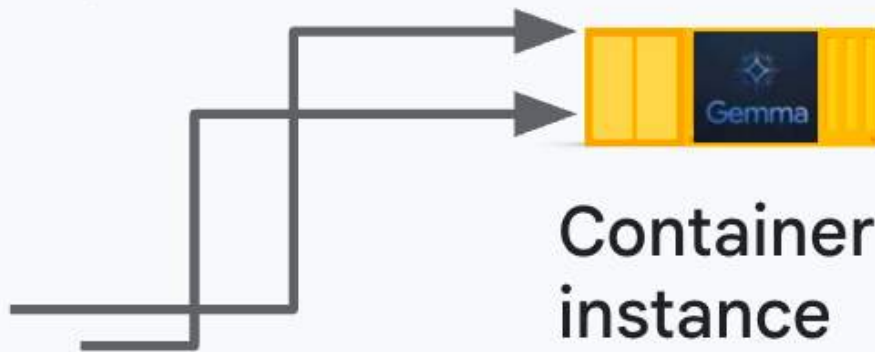
Tune this value based on the performance of the specific AI inference framework and implementation considering these factors:

- Number of model instances loaded onto the GPU
- Number of parallel queries per model
- Use of batching
- Amount of non-GPU work



Service revision

Concurrent requests



Concurrent requests to an instance

Setting the maximum concurrency

A general rule of thumb is to use the formula: $(N * P) + (N * B)$ where:

- N is the number of model instances.
- P is the parallel queries per model.
- B is the ideal batch size.

For example, if one model instance is loaded onto the GPU ($N=1$), and each model instance can handle four parallel queries ($P=4$), the ideal batch size is also four ($B=4$), you would set the maximum concurrent requests per instance to: $(1 * 4) + (1 * 4) = 8$.



Note that the formula above is a rule of thumb, with the ideal setting depending on your specific implementation. To achieve actual optimal performance, it is recommended to load test your service with different max concurrency settings and evaluate performance.

What happens if:

Maximum concurrent requests is set too high

Requests might end up queuing in the instance waiting for access to the GPU leading to increased request latency.

Maximum concurrent requests is set too low

GPU might be underutilized causing Cloud Run to scale out more instances than necessary.

Using startup probes

For container health checks, you configure startup and liveness probes for your Cloud Run services. A startup probe determines if a container instance is started and ready to accept traffic. By default, a TCP startup probe is automatically configured for a new Cloud Run service with default values.

Startup probe



Container instance

Startup probe

Consider these factors when configuring a startup probe for your AI inference service on Cloud Run:

- Allow adequate time for your model to load and initialize, and for the container to startup.
- Configure the probe to pass only when the application is ready to serve requests.

For more information, view the documentation on [configuring container health checks](#) on Cloud Run.

Runtime recommendations

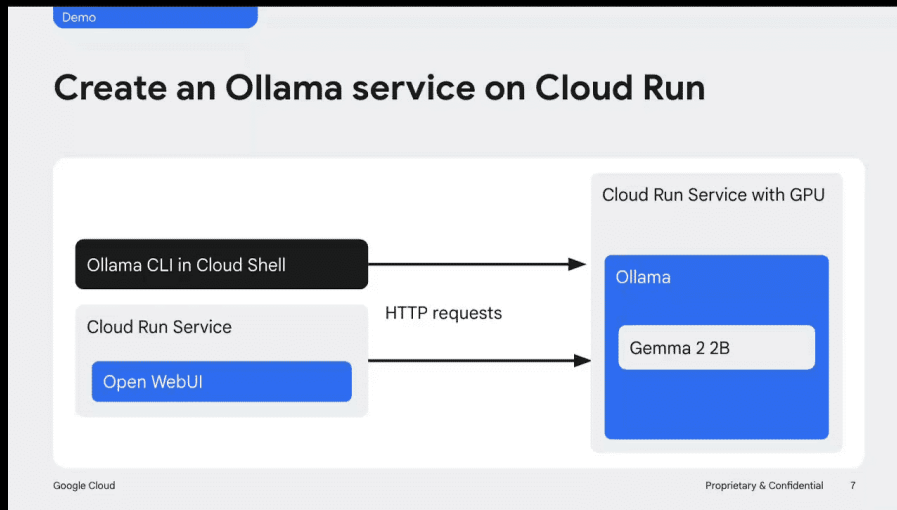
Here are some recommendations to manage your AI inference service on Cloud Run at runtime:

- Manage your supported context length. The smaller the context window, the more queries you can support running in parallel.
- Use the model caches that were generated at build time.
- If supported by the framework, use a quantized key-value cache to reduce memory requirements per query.
- Tune the amount of GPU memory to reserve for model weights, activation, and key-value caches.
- Make sure your application code can handle your configured Cloud Run service concurrency settings.

Demo

In this video, we deploy an Ollama service on Cloud Run with GPU. The service performs AI inference using the Gemma open-source model that is stored in Cloud Storage and accessed using a volume mount configured for the Cloud Run service.

Click the play button to watch the video.



Knowledge check

Match the model storage location to its characteristics.

...	The model is stored in Cloud Storage.	Reduces the time taken to start the container instance.
...	The model is included in the container image.	Enables faster deployment of the container image to Cloud Run.
...	The model is downloaded from the internet.	Route all traffic through the VPC network, and set up Cloud NAT with high bandwidth.

SUBMIT

Integrate AI inference Services

This lesson discusses how to integrate Cloud Run services with Google Cloud database and other services for running AI inference.

Database services

Google Cloud offers various managed database services that can be used to store and manage relational and non-relational data. Some of the relational databases can also be used as vector databases for gen AI applications.

Click each tab to learn more.

RELATIONAL DATABASES

NON-RELATIONAL DATABASES

VECTOR DATABASES

Relational databases store and organize data in tables with defined relationships.

- Cloud SQL: A fully managed database service for MySQL, PostgreSQL, and SQL Server.
- Spanner: A globally consistent database with virtually unlimited scale.
- AlloyDB: A PostgreSQL fully compatible database.



Cloud SQL



Spanner



AlloyDB

RELATIONAL DATABASES

NON-RELATIONAL DATABASES

VECTOR DATABASES

Non-relational databases, also known as NoSQL databases, store data in a flexible non-tabular format.

- Bigtable: A highly flexible NoSQL database that is used for high throughput, low-latency applications at scale.
- Memorystore: A fully managed in-memory database service.
- Firestore: A highly scalable and available serverless document database.



RELATIONAL DATABASES	NON-RELATIONAL DATABASES	VECTOR DATABASES
Vector databases enable you to store, index, and query vector embeddings--numerical representations of unstructured data such as text, images, and audio. These databases enable efficient searches for semantic similarity. • AlloyDB AI: Build gen AI applications with AlloyDB for PostgreSQL working with vectors, models, and data. • Spanner: Store data, and vectors in Spanner for full-text, and vector search. • Cloud SQL: Use MySQL or PostgreSQL to store and search vectors, and use pgvector to quickly build gen AI applications. AlloyDB Spanner Cloud SQL		

Vector databases enable you to store, index, and query vector embeddings--numerical representations of unstructured data such as text, images, and audio. These databases enable efficient searches for semantic similarity.

- AlloyDB AI: Build gen AI applications with AlloyDB for PostgreSQL working with vectors, models, and data.
- Spanner: Store data, and vectors in Spanner for full-text, and vector search.
- Cloud SQL: Use MySQL or PostgreSQL to store and search vectors, and use pgvector to quickly build gen AI applications.



Design patterns

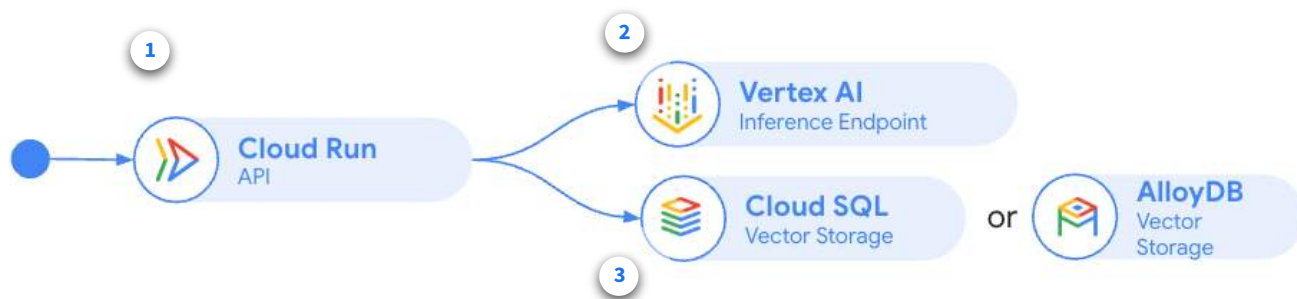
In the previous section, you learned how to host a model on Cloud Run.

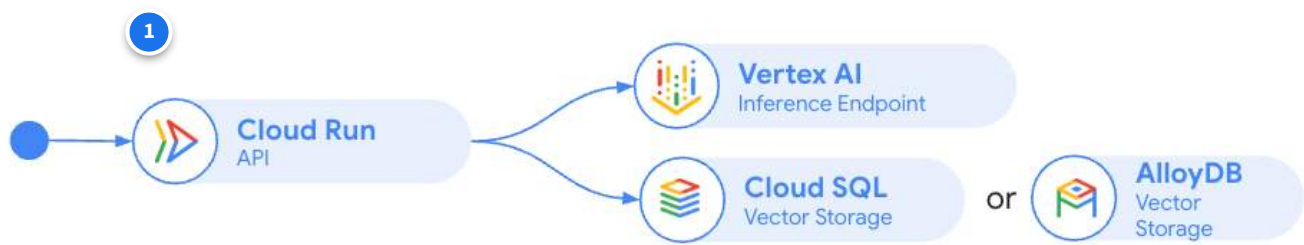
This is one possible design pattern. In this section, you'll learn about other design patterns including how to use models available on Vertex AI for inference.

Gen AI apps and AI agents

You can use Cloud Run to provide a scalable API endpoint that runs popular gen AI app frameworks such as LangChain or Firebase Genkit. The Cloud Run app integrates with Vertex AI for model inferencing, and uses Cloud SQL or AlloyDB as a vector database.

Click each marker to learn more.

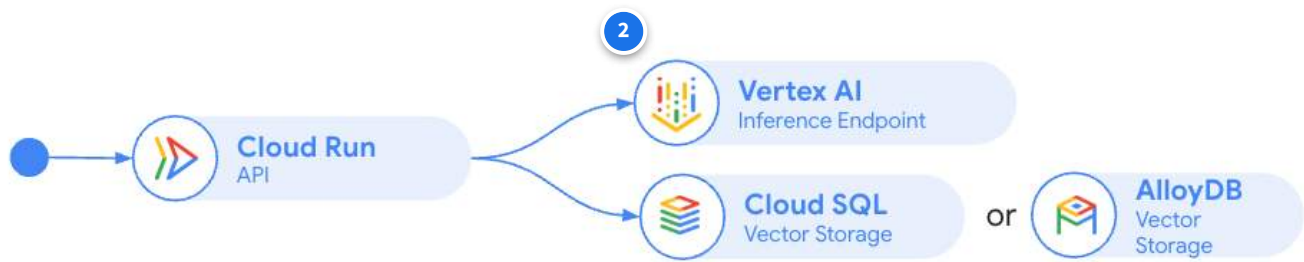




Serve

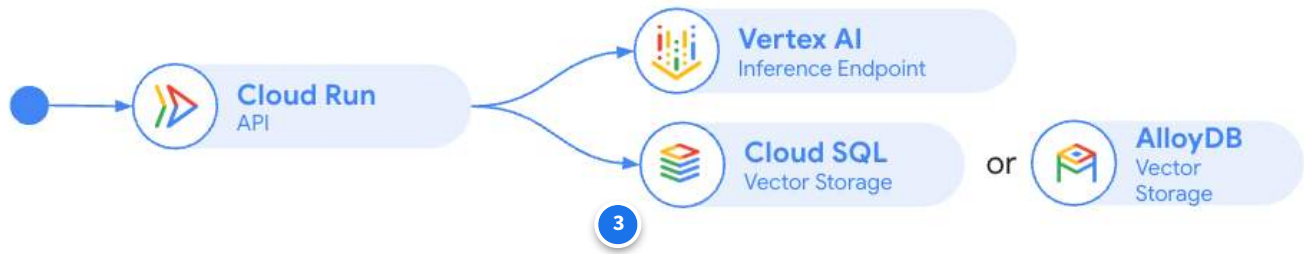
Use Cloud Run to provide a scalable API endpoint to serve requests to your gen AI app.

You can run LangChain, Firebase Genkit, or custom code to validate requests and orchestrate calls to models.



Inference

To invoke an AI model, you can use Vertex AI.



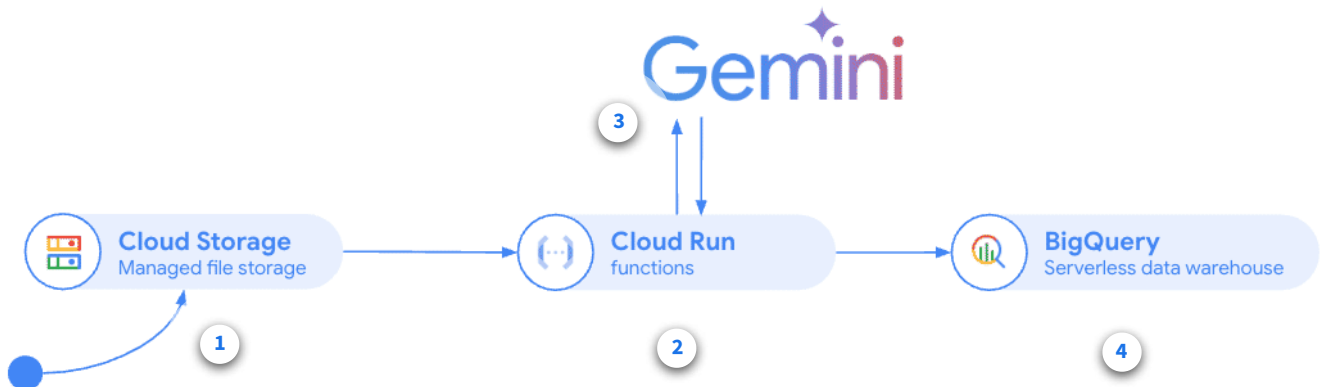
Vector database

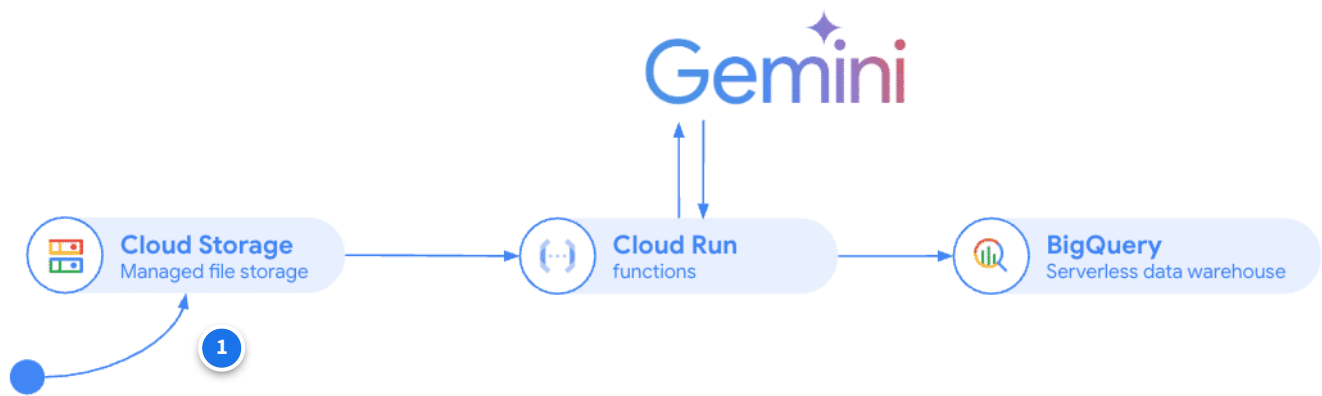
To store vector embeddings for AI inference, you can use Cloud SQL or AlloyDB and the pgvector PostgreSQL extension.

Data processing

You can use Cloud Run functions to process data in response to a Cloud Event. The function can invoke the Gemini API to extract relevant information from the data and store it in a database such as BigQuery.

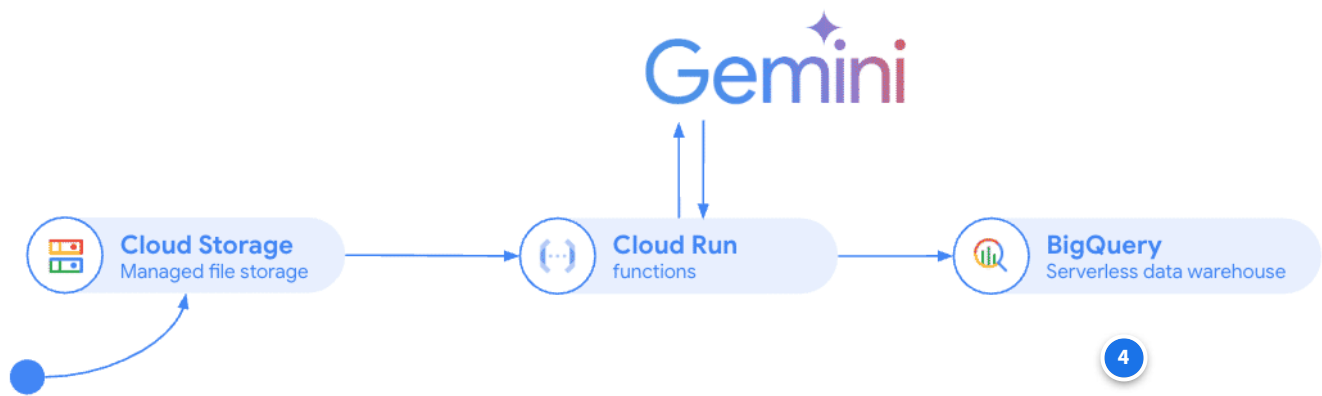
Click each marker to learn more.





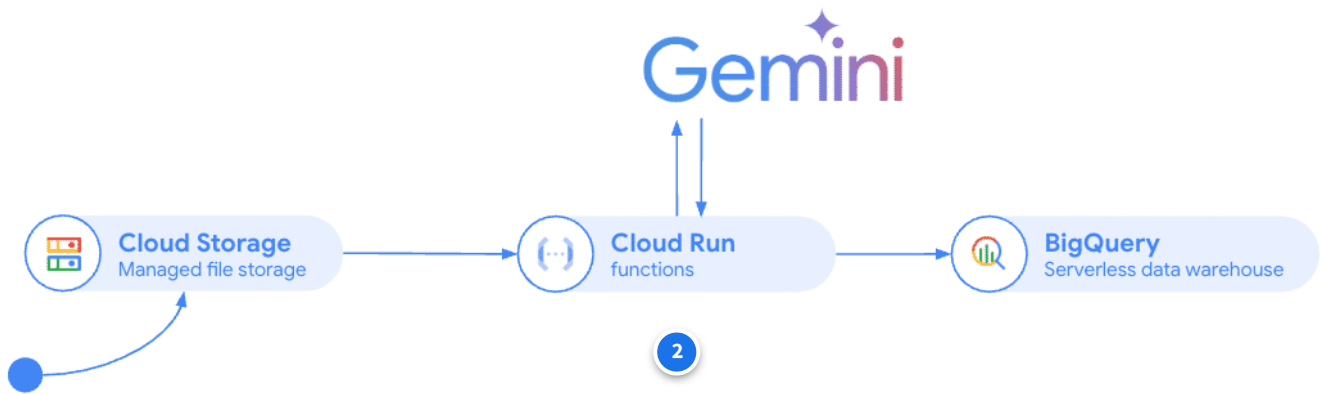
Cloud Event

A cloud event is generated when a file is created in Cloud Storage.



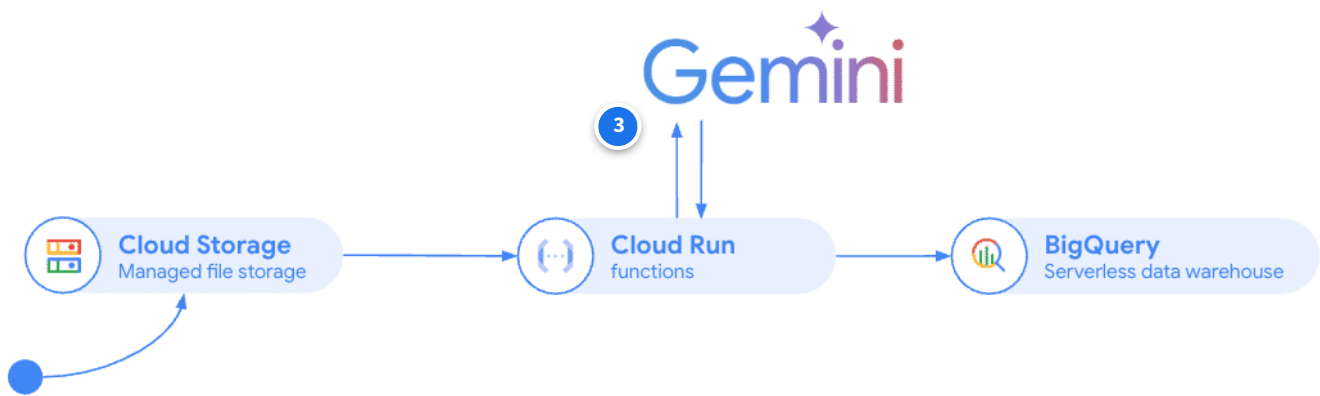
BigQuery

The Cloud Run function then processes the Gemini response and saves it to BigQuery.



Cloud Run function

To process the event, a Cloud Run function is triggered.



Gemini API

The Cloud Run function invokes the Gemini API to extract information from data in the file.

LLM tuning and batch inferencing

You can use Cloud Run GPU jobs to tune a model and perform batch inferencing, and store the results in Cloud Storage.

Click each marker to learn more.

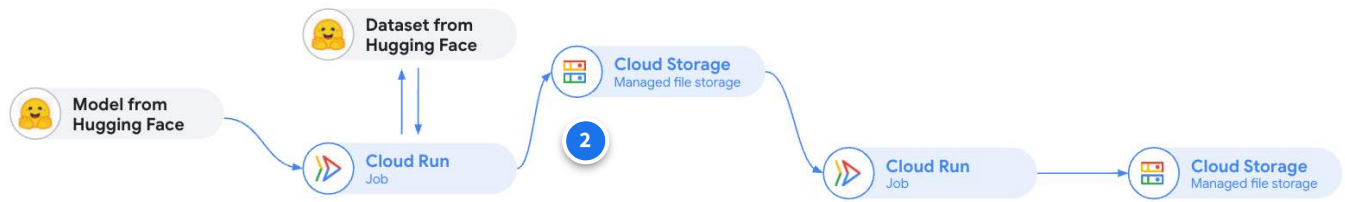




Load

Load a base LLM from a model provider such as Hugging Face.

[Hugging Face](#) is a platform where users can collaborate on building and using AI models.



Tune

Use a Cloud Run GPU job to tune the model using a training dataset from Hugging Face or Cloud Storage.

Store the tuned model in Cloud Storage.



Batch inference

Perform batch-inference with a Cloud Run GPU job using the tuned model and batched input data.

Batch inference is the asynchronous process of generating model results on a batch of input data, and storing the results in a datastore for use later.



Store results

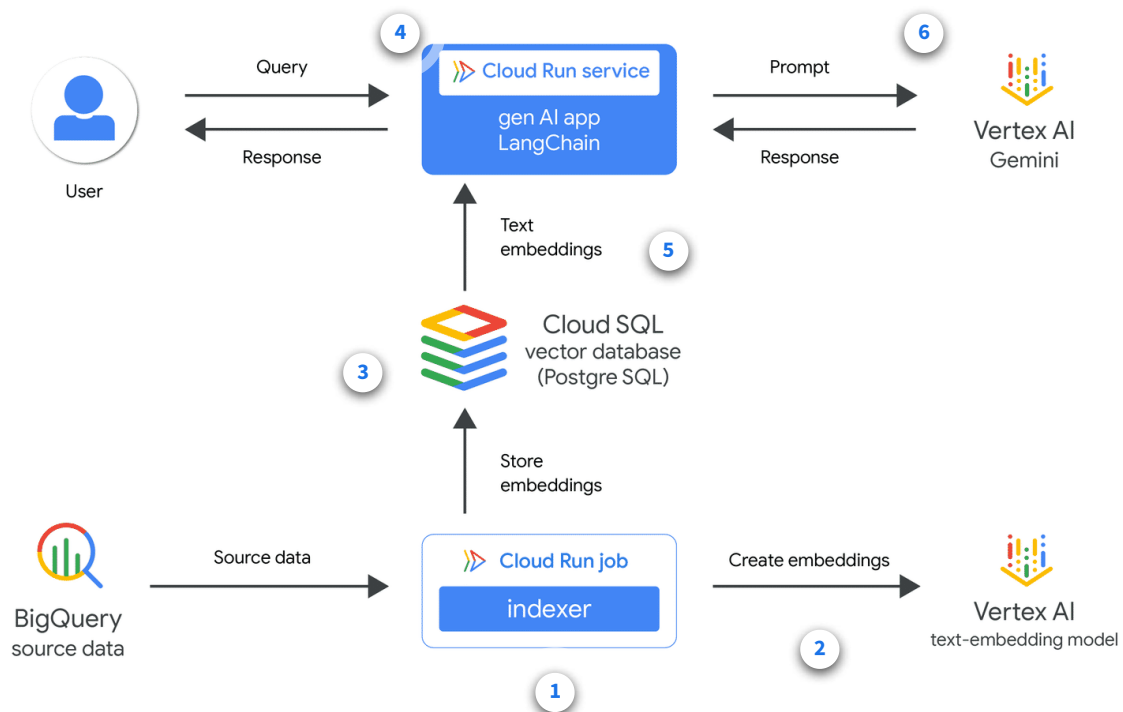
Store the model results in Cloud Storage.

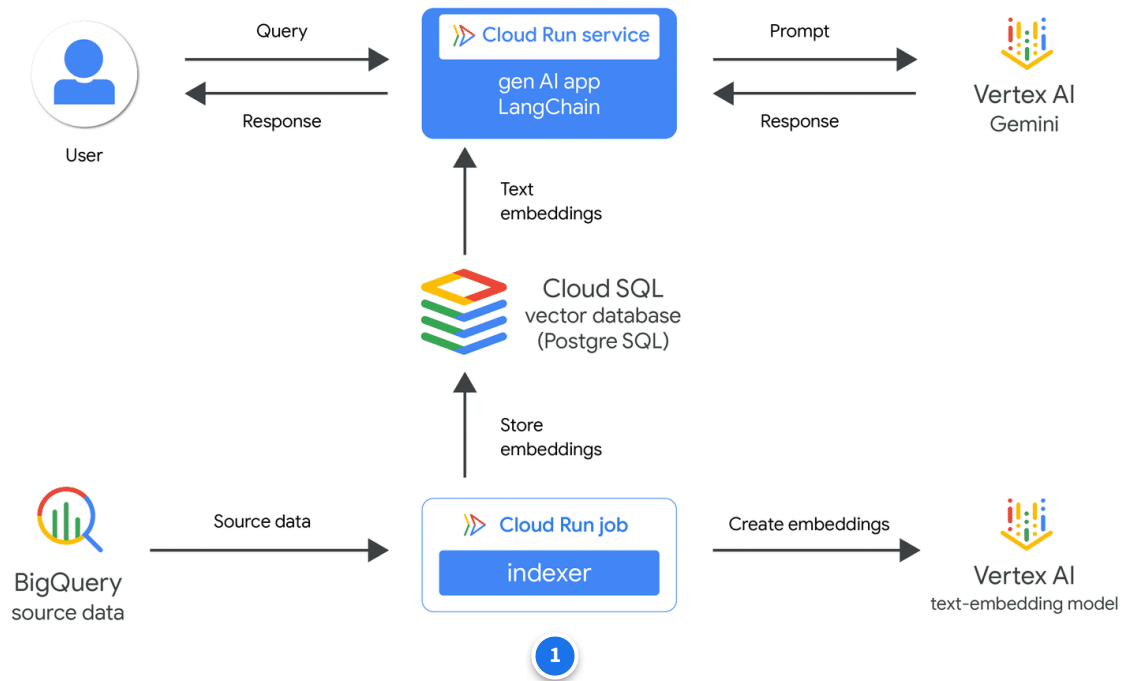
Develop a gen AI app on Cloud Run

Let's review how you can deploy a gen AI app that uses LangChain and Gemini to Cloud Run.

The app integrates with a PostgreSQL vector database to store embeddings, and with Vertex AI for model inferencing.

Click each marker to learn more.

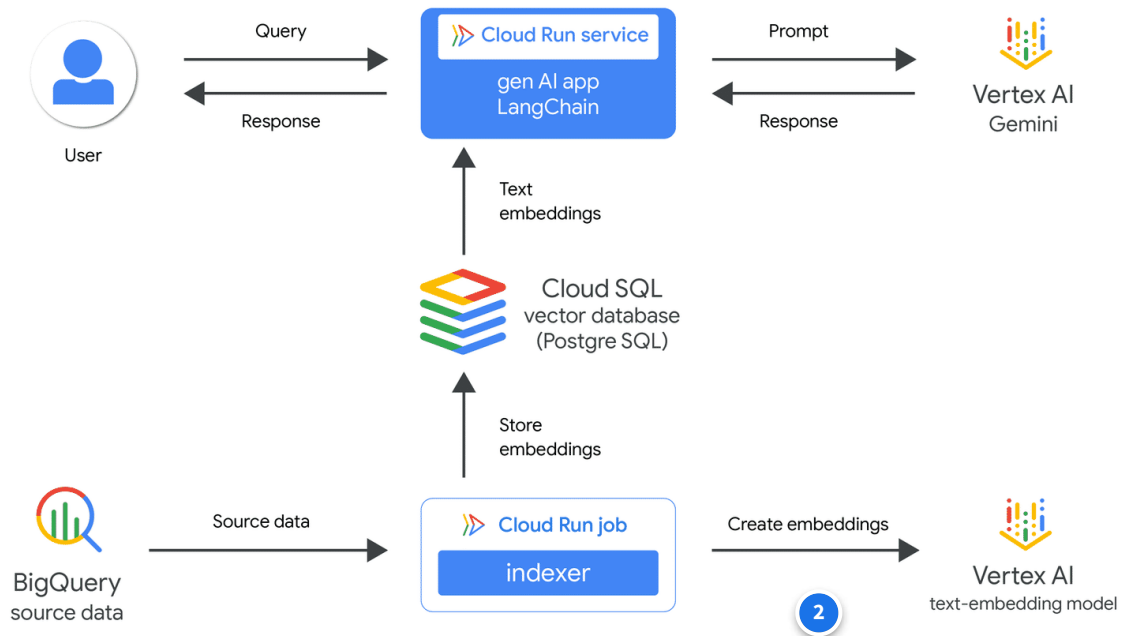




Indexer

In this step, you create an indexer that takes your source data from BigQuery or other database and converts the data into vectors using a text embedding model.

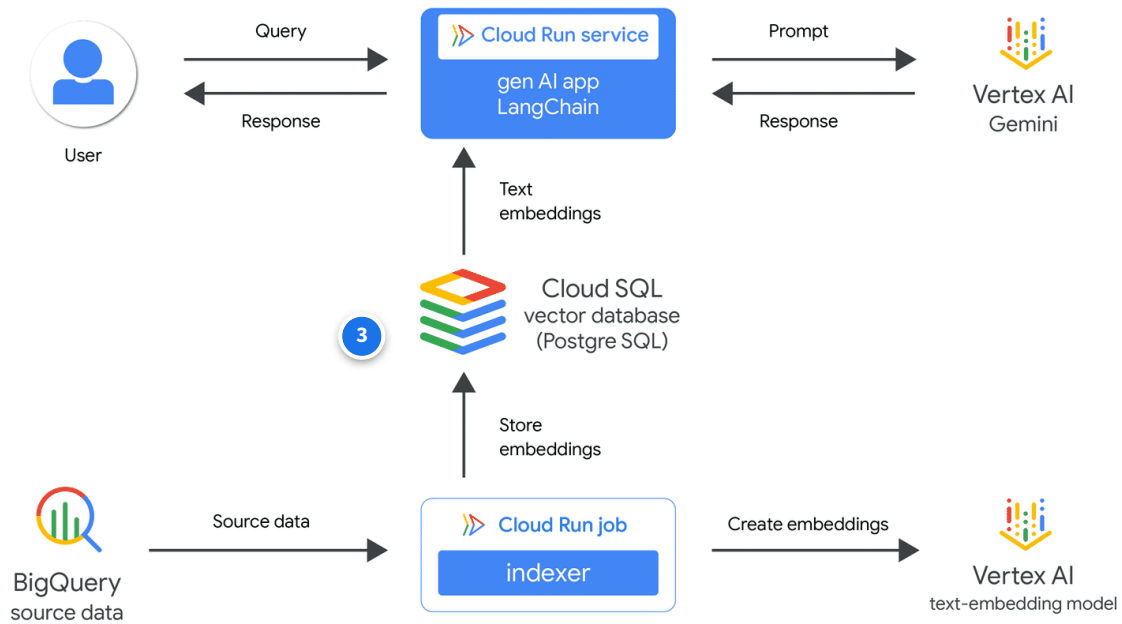
The indexer is deployed to Cloud Run as a job.



Compute embeddings

To perform the indexing, you compute embeddings by integrating a text embedding model from Vertex AI on Google Cloud.

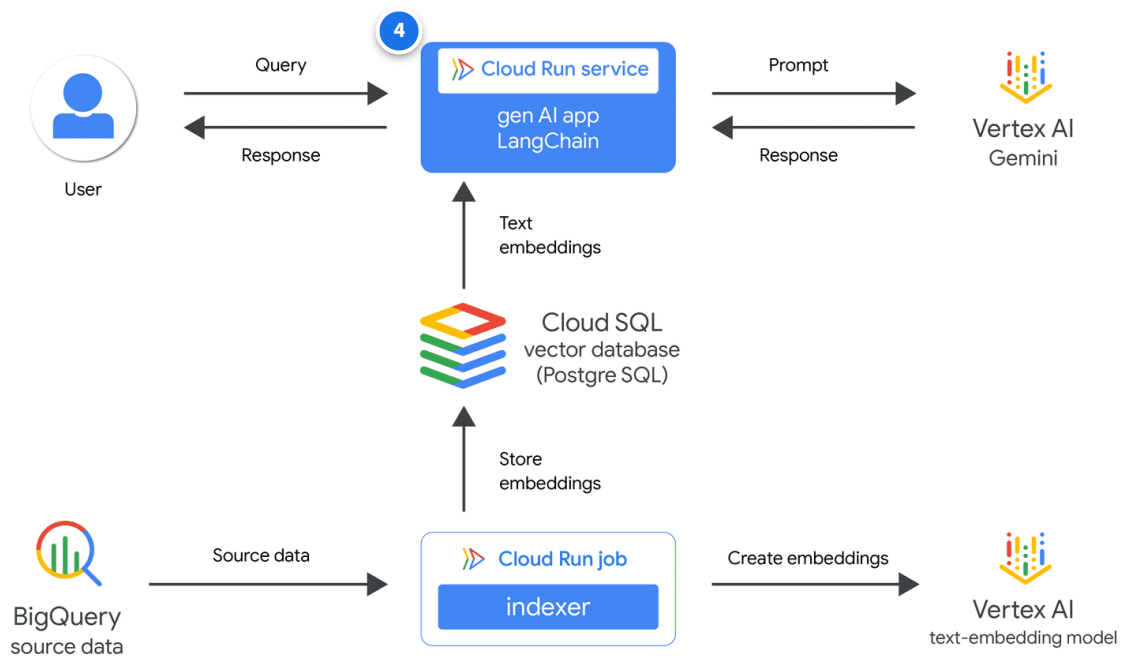
An embedding is a numerical vector that captures the meaning and context of words in the text data.



Store embeddings

The text embeddings are stored in a vector database, in this case, PostgreSQL with pgvector using Cloud SQL on Google Cloud.

pgvector is PostgreSQL extension that enables you to store, query, and index vectors in the database.

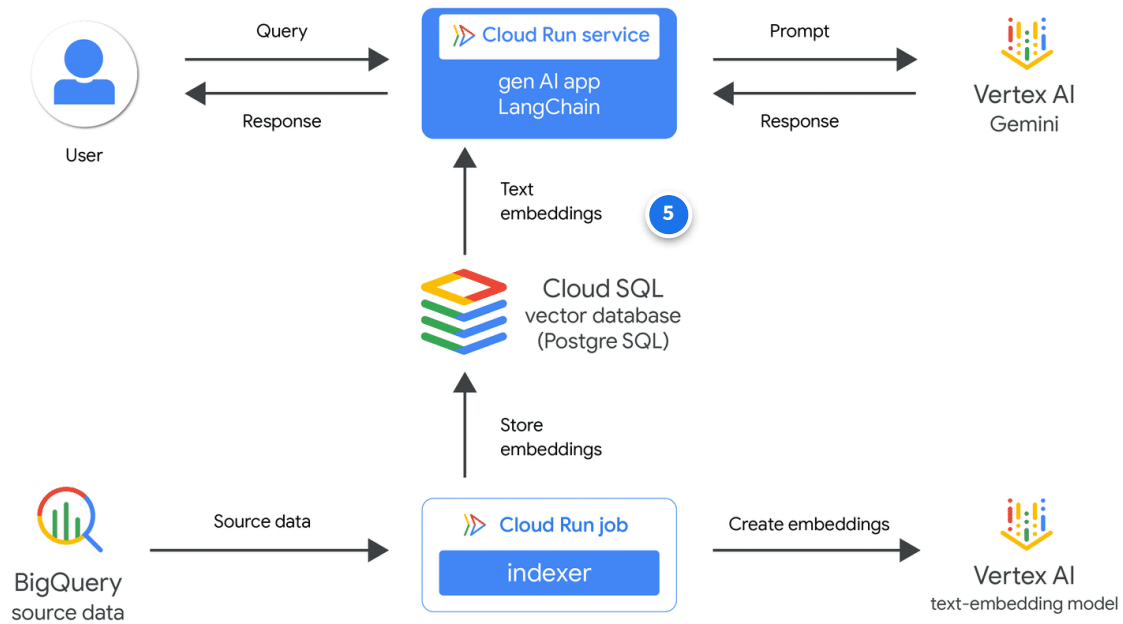


Gen AI app

The gen AI app uses the pgvector extension to connect to the PostgreSQL vector database.

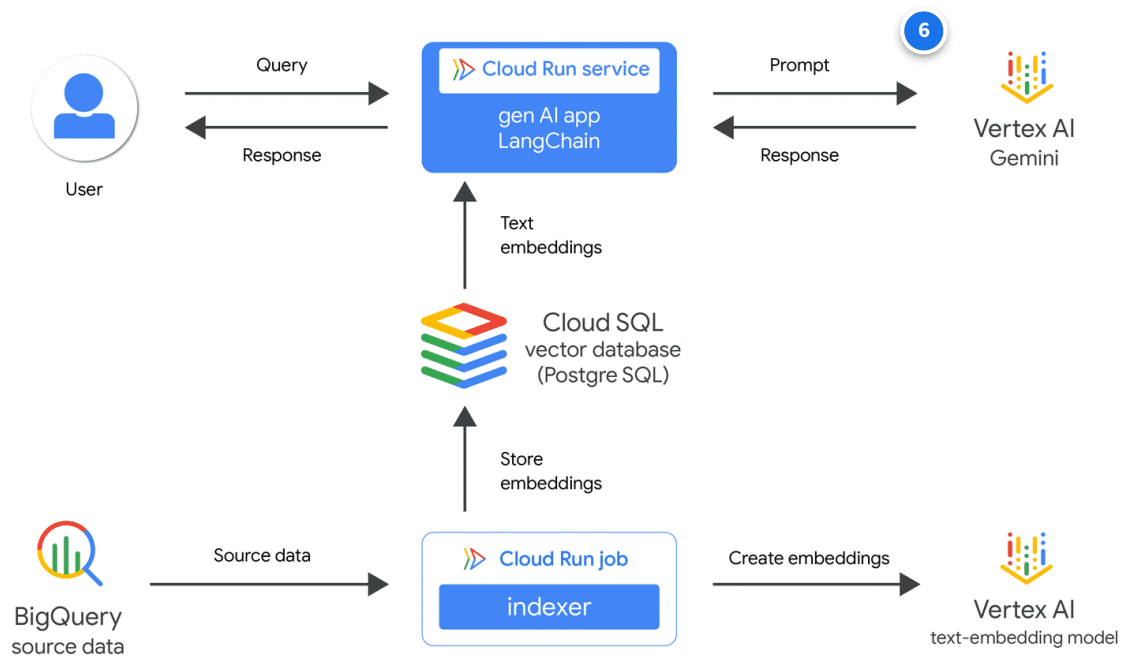
The app uses LangChain, a framework for building GenAI applications by enabling you to chain together different models and tools.

The app is deployed to Cloud Run as a service.



Retriever

The app creates a retriever to fetch the text embeddings from the database.



Model invocation

The app uses LangChain to integrate with the Gemini family of models that is part of the Vertex AI platform on Google Cloud.

The model invocation includes a prompt template that contains the retrieved text embeddings, and the user query.

The video below demonstrates the process to deploy a gen AI app using LangChain on Cloud Run. It shows how you can use Cloud SQL as a vector database to store embeddings and use a Gemini model in Vertex AI for inference.

Click the play button to watch the video.

Knowledge check

Which of the following is a primary characteristic of vector databases?

- ☐ Storing data as high-dimensional numerical vectors.
- ☐ Directly executing SQL queries on the vector data.
- ☐ Enabling efficient searches for semantic similarity.

SUBMIT

What did I walk away with?

Congratulations on completing this course. Here are some of the key points to remember:

- ✓ To run AI inference on Cloud Run, configure your service with GPUs.
- ✓ Use lightweight language models on Cloud Run for AI inference.
- ✓ Store lightweight models in the container image and larger models in Cloud Storage while running AI inference on Cloud Run.
- ✓ Integrate your gen AI app on Cloud Run with vector databases on Google Cloud.



Additional resources

This course included content and practical demos to show how you can deploy models and run AI inference services on Cloud Run. To learn more and get experience with using Cloud Run for AI inference, run the codelabs and read the documentation using these links:

Lab

Run AI Inference on Cloud Run with GPUs and vLLM.

[CODELAB](#)

Lab

Use Ollama with Cloud Run GPUs and Open WebUI.

CODELAB

Lab

Build and Deploy a LangChain App on Cloud Run.

CODELAB

Review

Read documentation on GPU support for Cloud Run services.

DOCUMENTATION

Congratulations on completing this training!



To earn credits, please return to Google Cloud Skills Boost and complete the quiz.

